

```
        return QueryResult(sought, nodata, file);           // 没有找到
    else
        return QueryResult(sought, loc->second, file);
}
```

和过去一样，返回类型不在类的作用域中（参见 7.4 节，第 253 页），因此我们必须指明函数的返回值是 `TextQuery::QueryResult` 类型。不过在函数体内部我们可以直接访问 `QueryResult`，比如上面的 `return` 语句就是这样。

嵌套类和外层类是相互独立的

尽管嵌套类定义在其外层类的作用域中，但是读者必须谨记外层类的对象和嵌套类的对象没有任何关系。嵌套类的对象只包含嵌套类定义的成员；同样，外层类的对象只包含外层类定义的成员，在外层类对象中不会有任何嵌套类的成员。

说得再具体一些，`TextQuery::query` 的第二条 `return` 语句

```
return QueryResult(sought, loc->second, file);
```

使用了 `TextQuery` 对象的数据成员，而 `query` 正是用它们来初始化 `QueryResult` 对象的。因为在一个 `QueryResult` 对象中不包含其外层类的成员，所以我们必须使用上述成员构造我们返回的 `QueryResult` 对象。

◀ 847

19.5 节练习

练习 19.20：将你的 `QueryResult` 类嵌套在 `TextQuery` 中，然后重新运行 12.3.2 节（第 435 页）中使用了 `TextQuery` 的程序。

19.6 union：一种节省空间的类

联合（union）是一种特殊的类。一个 union 可以有多个数据成员，但是在任意时刻只有一个数据成员可以有值。当我们给 union 的某个成员赋值之后，该 union 的其他成员就变成未定义的状态了。分配给一个 union 对象的存储空间至少要能容纳它的最大的数据成员。和其他类一样，一个 union 定义了一种新类型。

类的某些特性对 union 同样适用，但并非所有特性都如此。union 不能含有引用类型的成员，除此之外，它的成员可以是绝大多数类型。在 C++11 新标准中，含有构造函数或析构函数的类类型也可以作为 union 的成员类型。union 可以为其成员指定 `public`、`protected` 和 `private` 等保护标记。默认情况下，union 的成员都是公有的，这一点与 `struct` 相同。

union 可以定义包括构造函数和析构函数在内的成员函数。但是由于 union 既不能继承自其他类，也不能作为基类使用，所以在 union 中不能含有虚函数。

定义 union

union 提供了一种有效的途径使得我们可以方便地表示一组类型不同的互斥值。举个例子，假设我们需要处理一些不同种类的数字数据和字符数据，则在此过程中可以定义一个 union 来保存这些值：

```
// Token 类型的对象只有一个成员，该成员的类型可能是下列类型中的任意一种
union Token {
```

```
// 默认情况下成员是公有的
char   cval;
int    ival;
double dval;

};
```

在定义一个 union 时，首先是关键字 union，随后是该 union 的（可选的）名字以及花括号内的一组成员声明。上面的代码定义了一个名为 Token 的 union，它可以保存一个值，这个值的类型可能是 char、int 或 double 中的一种。

848 使用 union 类型

union 的名字是一个类型名。和其他内置类型一样，默认情况下 union 是未初始化的。我们可以像显式地初始化聚合类（参见 7.5.5 节，第 266 页）一样使用一对花括号内的初始值显式地初始化一个 union：

```
Token first_token = {'a'};           // 初始化 cval 成员
Token last_token;                     // 未初始化的 Token 对象
Token *pt = new Token;                // 指向一个未初始化的 Token 对象的指针
```

如果提供了初始值，则该初始值被用于初始化第一个成员。因此，first_token 的初始化过程实际上是给 cval 成员赋了一个初值。

我们使用通用的成员访问运算符访问一个 union 对象的成员：

```
last_token.cval = 'z';
pt->ival = 42;
```

为 union 的一个数据成员赋值会令其他数据成员变成未定义的状态。因此，当我们使用 union 时，必须清楚地知道当前存储在 union 中的值到底是什么类型。如果我们使用错误的数据成员或者为错误的数据成员赋值，则程序可能崩溃或出现异常行为，具体的情况根据成员的类型而有所不同。

匿名 union

匿名 union (anonymous union) 是一个未命名的 union，并且在右花括号和分号之间没有任何声明（参见 2.6.1 节，第 65 页）。一旦我们定义了一个匿名 union，编译器就自动地为该 union 创建一个未命名的对象：

```
union {                               // 匿名 union
    char cval;
    int ival;
    double dval;
}; // 定义一个未命名的对象，我们可以直接访问它的成员
cval = 'c';                           // 为刚刚定义的未命名的匿名 union 对象赋一个新值
ival = 42;                             // 该对象当前保存的值是 42
```

在匿名 union 的定义所在的作用域内该 union 的成员都是可以直接访问的。



匿名 union 不能包含受保护的成员或私有成员，也不能定义成员函数。

含有类类型成员的 union

C++ 的早期版本规定，在 union 中不能含有定义了构造函数或拷贝控制成员的类型

成员。C++11 新标准取消了这一限制。不过, 如果 union 的成员类型定义了自己的构造函数和/或拷贝控制成员, 则该 union 的用法要比只含有内置类型成员的 union 复杂得多。

849

C++
11

当 union 包含的是内置类型的成员时, 我们可以使用普通的赋值语句改变 union 保存的值。但是对于含有特殊类类型成员的 union 就没这么简单了。如果我们想将 union 的值改为类类型成员对应的值, 或者将类类型成员的值改为一个其他值, 则必须分别构造或析构该类类型的成员: 当我们将 union 的值改为类类型成员对应的值时, 必须运行该类型的构造函数; 反之, 当我们将类类型成员的值改为一个其他值时, 必须运行该类型的析构函数。

当 union 包含的是内置类型的成员时, 编译器将按照成员的次序依次合成默认构造函数或拷贝控制成员。但是如果 union 含有类类型的成员, 并且该类型自定义了默认构造函数或拷贝控制成员, 则编译器将为 union 合成对应的版本并将其声明为删除的 (参见 13.1.6 节, 第 450 页)。

例如, string 类定义了五个拷贝控制成员以及一个默认构造函数。如果 union 含有 string 类型的成员, 并且没有自定义默认构造函数或某个拷贝控制成员, 则编译器将合成缺少的成员并将其声明为删除的。如果在某个类中含有一个 union 成员, 而且该 union 含有删除的拷贝控制成员, 则该类与之对应的拷贝控制操作也将是删除的。

使用类管理 union 成员

对于 union 来说, 要想构造或销毁类类型的成员必须执行非常复杂的操作, 因此我们通常把含有类类型成员的 union 内嵌在另一个类当中。这个类可以管理并控制与 union 的类类型成员有关的状态转换。举个例子, 我们为 union 添加一个 string 成员, 并将我们的 union 定义成匿名 union, 最后将它作为 Token 类的一个成员。此时, Token 类将可以管理 union 的成员。

为了追踪 union 中到底存储了什么类型的值, 我们通常会定义一个独立的对象, 该对象称为 union 的判别式 (discriminant)。我们可以使用判别式辨认 union 存储的值。为了保持 union 与其判别式同步, 我们将判别式也作为 Token 的成员。我们的类将定义一个枚举类型 (参见 19.3 节, 第 736 页) 的成员来追踪其 union 成员的状态。

在我们的类中定义的函数包括默认构造函数、拷贝控制成员以及一组赋值运算符, 这些赋值运算符可以将 union 的某种类型的值赋给 union 成员:

```
class Token {
public:
    // 因为 union 含有一个 string 成员, 所以 Token 必须定义拷贝控制成员
    // 定义移动构造函数和移动赋值运算符的任务留待本节练习完成
    Token(): tok(INT), ival{0} { }
    Token(const Token &t): tok(t.tok) { copyUnion(t); }
    Token &operator=(const Token&);
    // 如果 union 含有一个 string 成员, 则我们必须销毁它, 参见 19.1.2 节 (第 729 页)
    ~Token() { if (tok == STR) sval.~string(); }
    // 下面的赋值运算符负责设置 union 的不同成员
    Token &operator=(const std::string&);
    Token &operator=(char);
    Token &operator=(int);
    Token &operator=(double);
private:
```

850

```
enum {INT, CHAR, DBL, STR} tok; // 判别式
union { // 匿名 union
    char    cval;
    int     ival;
    double  dval;
    std::string sval;
}; // 每个 Token 对象含有一个该未命名 union 类型的未命名成员
// 检查判别式, 然后酌情拷贝 union 成员
void copyUnion(const Token&);
};
```

我们的类定义了一个嵌套的、未命名的、不限定作用域的枚举类型（参见 19.3 节，第 736 页），并将其作为 tok 成员的类型。其中，tok 的声明位于枚举类型定义的右侧花括号之后，以及表示该枚举类型定义结束的分号之前，因此，tok 的类型就是当前这个未命名的 enum 类型（参见 2.6.1 节，第 65 页）。

我们使用 tok 作为判别式。当 union 存储的是一个 int 值时，tok 的值是 INT；当 union 存储的是一个 string 值时，tok 的值是 STR；以此类推。

类的默认构造函数初始化判别式以及 union 成员，令其保存 int 值 0。

因为我们的 union 含有一个定义了析构函数的成员，所以必须为 union 也定义一个析构函数以销毁 string 成员。和普通的类类型成员不一样，作为 union 组成部分的类成员无法自动销毁。因为析构函数不清楚 union 存储的值是什么类型，所以它无法确定应该销毁哪个成员。

我们的析构函数检查被销毁的对象中是否存储着 string 值。如果有，则类的析构函数显式地调用 string 的析构函数（参见 19.1.2 节，第 729 页）释放该 string 使用的内存；反之，如果 union 存储的值是内置类型，则类的析构函数什么也不做。

管理判别式并销毁 string

类的赋值运算符将负责设置 tok 并为 union 的相应成员赋值。和析构函数一样，这些运算符在为 union 赋新值前必须首先销毁 string：

```
Token &Token::operator=(int i)
{
    if (tok == STR) sval.~string(); // 如果当前存储的是 string, 释放它
    ival = i;                       // 为成员赋值
    tok = INT;                     // 更新判别式
    return *this;
}
```

如果 union 的当前值是 string，则我们必须先调用 string 的析构函数销毁这个 string，然后再为 union 赋新值。清除了 string 成员之后，我们将给定的值赋给与运算符形参类型相匹配的成员。在此例中，形参类型是 int，所以我们赋值给 ival。随后更新判别式并返回结果。

double 和 char 版本的赋值运算符与 int 赋值运算符非常相似，读者可以在本节的练习中尝试使用这两个运算符。string 版本与其他几个有所区别，原因是 string 版本必须管理与 string 类型有关的转换：

```
Token &Token::operator=(const std::string &s)
{
```

```

    if (tok == STR)                // 如果当前存储的是 string, 可以直接赋值
        sval = s;
    else
        new(&sval) string(s);      // 否则需要先构造一个 string
    tok = STR;                    // 更新判别式
    return *this;
}

```

在此例中, 如果 union 当前存储的是 string, 则我们可以使用普通的 string 赋值运算符直接为其赋值。如果 union 当前存储的不是 string, 则我们找不到一个已存在的 string 对象供我们调用赋值运算符。此时, 我们必须先利用定位 new 表达式 (参见 19.1.2 节, 第 729 页) 在内存中为 sval 构造一个 string, 然后将该 string 初始化为 string 形参的副本, 最后更新判别式并返回结果。

管理需要拷贝控制的联合成员

和依赖于类型的赋值运算符一样, 拷贝构造函数和赋值运算符也需要先检验判别式以明确拷贝所采用的方式。为了完成这一任务, 我们定义一个名为 copyUnion 的成员。

当我们在拷贝构造函数中调用 copyUnion 时, union 成员将被默认初始化, 这意味着编译器会初始化 union 的第一个成员。因为 string 不是第一个成员, 所以显然 union 成员保存的不是 string。在赋值运算符中情况有些不一样, 有可能 union 已经存储了一个 string。我们将在赋值运算符中直接处理这种情况。copyUnion 假设如果它的形参存储了 string, 则它一定会构造自己的 string:

```

void Token::copyUnion(const Token &t)
{
    switch (t.tok) {
        case Token::INT: ival = t.ival; break;
        case Token::CHAR: cval = t.cval; break;
        case Token::DBL: dval = t.dval; break;
        // 要想拷贝一个 string 可以使用定位 new 表达式构造它, 参见 19.1.2 节 (第 729 页)
        case Token::STR: new(&sval) string(t.sval); break;
    }
}

```

该函数使用一个 switch 语句 (参见 5.3.2 节, 第 159 页) 检验判别式。对于内置类型来说, 我们把值直接赋给对应的成员; 如果拷贝的是一个 string, 则需要构造它。 ◀ 852

赋值运算符必须处理 string 成员的三种可能情况: 左侧运算对象和右侧运算对象都是 string、两个运算对象都不是 string、只有一个运算对象是 string:

```

Token &Token::operator=(const Token &t)
{
    // 如果此对象的值是 string 而 t 的值不是, 则我们必须释放原来的 string
    if (tok == STR && t.tok != STR) sval.~string();
    if (tok == STR && t.tok == STR)
        sval = t.sval; // 无须构造一个新 string
    else
        copyUnion(t); // 如果 t.tok 是 STR, 则需要构造一个 string
    tok = t.tok;
    return *this;
}

```

如果作为左侧运算对象的 union 的值是 string 但右侧运算对象的值不是, 则我们必须先释放原来的 string 再给 union 成员赋一新值。如果两侧运算对象的值都是 string, 则我们可以使用普通的 string 赋值运算符完成拷贝。否则, 我们调用 copyUnion 进行赋值。在 copyUnion 内部, 如果右侧运算对象是 string, 则我们在左侧运算对象的 union 成员内构造一个新 string; 如果两端都不是 string, 则直接执行普通的赋值操作就可以了。

19.6 节练习

练习 19.21: 编写你自己的 Token 类。

练习 19.22: 为你的 Token 类添加一个 Sales_data 类型的成员。

练习 19.23: 为你的 Token 类添加移动构造函数和移动赋值运算符。

练习 19.24: 如果我们将一个 Token 对象赋给它自己将发生什么情况?

练习 19.25: 编写一系列赋值运算符, 令其分别接受 union 中各种类型的值。

19.7 局部类

类可以定义在某个函数的内部, 我们称这样的类为**局部类** (local class)。局部类定义的类型只在定义它的作用域内可见。和嵌套类不同, 局部类的成员受到严格限制。



局部类的所有成员 (包括函数在内) 都必须完整定义在类的内部。因此, 局部类的作用与嵌套类相比相差很远。

853

在实际编程的过程中, 因为局部类的成员必须完整定义在类的内部, 所以成员函数的复杂性不可能太高。局部类的成员函数一般只有几行代码, 否则我们就很难读懂它了。

类似的, 在局部类中也不允许声明静态数据成员, 因为我们没法定义这样的成员。

局部类不能使用函数作用域中的变量

局部类对其外层作用域中名字的访问权限受到很多限制, 局部类只能访问外层作用域定义的类型名、静态变量 (参见 6.1.1 节, 第 185 页) 以及枚举成员。如果局部类定义在某个函数内部, 则该函数的普通局部变量不能被该局部类使用:

```
int a, val;
void foo(int val)
{
    static int si;
    enum Loc { a = 1024, b };
    // Bar 是 foo 的局部类
    struct Bar {
        Loc locVal;                // 正确: 使用一个局部类型名
        int barVal;

        void fooBar(Loc l = a)     // 正确: 默认实参是 Loc::a
        {
            barVal = val;         // 错误: val 是 foo 的局部变量
        }
    };
}
```